EMA: Technical Report

A THEORETICAL GUARANTEES OF EMA

In this section, we show that EMA preserves the asymptotic complexity of the base index in terms of both construction time and index size. We also derive the local deletion-work bound stated in Section 4.1 of the main paper. We further analyze the source of false positives introduced by Markers and provide theoretical bounds on the false positive rate.

A.1 Complexity

Lemma A.1 (Cost of MEncode and edge existance). *In Algorithm 3,* `MEncode` *takes $O(m)$ time. Testing whether $e_{(n,v)}$ previously exists takes $O(M)$ time, since it scans at most M previously stored neighbors.*

Theorem A.2. *The time complexity of Algorithm 3 is*

$$O(efc \cdot M \cdot (M+m)) .$$

Proof. The outer loop iterates over candidates $v \in Cand$, so it executes at most $|Cand| \le efc$ times.

For a fixed candidate v, the inner loop scans $u \in Nbrs$. The neighbor list is bounded by the maximal size M, hence $|Nbrs| \le M$ and the inner loop runs at most M times.

Consider one inner-loop iteration for a pair (u, v). Besides a constant-time distance comparison, when u dominates v the algorithm (i) tests whether $e_{(n,v)}$ previously exists, which costs $O(M)$, and then (ii) calls either Marker merge or `MEncode`, which costs $O(m)$ by Lemma A.1. Therefore, each inner-loop iteration costs $O(M+m)$ in the worst case.

Thus, for each candidate v, the inner loop costs

$$O(M \cdot (M+m)) .$$

The remaining work per candidate (insertion into $Nbrs$ and one additional `MEncode`) is at most $O(M+m)$ and is dominated by the inner-loop bound. Summing over all $|Cand| \le efc$ candidates gives

$$O(efc \cdot M \cdot (M+m)) .$$

□

EMA uses an upper vector-navigation layer (layer 1) and a bottom Marker-bearing layer (layer 0). Under the parameterized graph-search model, candidate search examines $O(efc \log n)$ rows per insertion, at cost $O(M \cdot efc \log n)$. Charging an additional $O(M+m)$ factor for edge checks and Marker operations gives $O(M \cdot (M+m) \cdot efc \cdot n \log n)$ total construction work. For fixed M, efc, and m, this is $O(n \log n)$.

Theorem A.3. *The space complexity of EMA is*

$$O(nM \cdot sm),$$

Proof. The additional storage introduced by EMA consists of the Marker stored per neighbor (edge), which is only used in the bottom layer.

In the bottom layer, each of the n nodes stores at most M neighbors, resulting in at most nM edges. Each edge stores a Marker of size sm bits, so the total Marker storage is

$$(nM) \cdot sm \text{ bits.}$$

The top layer contains at most n nodes, each with at most M outgoing edges. Its vector-navigation adjacency lists contribute $O(nM)$ space.

Combining both parts yields a total space complexity of

$$O(nM \cdot sm + nM) ,$$

which simplifies to $O(Mn \cdot sm)$. □

Deletion complexity. We analyze the local graph and Marker maintenance procedure for deleting one point a in the main paper's Marker-aware Deletion algorithm. The layer loop runs from 0 to level$(a) \in \{0, 1\}$; Marker maintenance applies on layer 0. Let n denote the number of stored points, including retained deleted records. We retain the degree parameter M, search width efc, and attribute count m. Vector dimension, per-attribute encoding capacities, and the reconnection budgets $k_r = 50$, $c = 3$ are fixed. Thus a distance evaluation costs $O(1)$, while a full Marker operation or `MEncode` costs $O(m)$. We use the parameterized search model of the construction analysis: navigation and each layer search examine $O(efc \log n)$ rows of degree $O(M)$, at cost $O(M \cdot efc \log n)$ including bounded-queue bookkeeping. The two-layer loop contributes a constant factor, while $\log n$ parameterizes the modeled graph-search work. Identifier-keyed work lists use the standard expected amortized $O(1)$ model for dictionary lookup and insertion. We count the logical graph and Marker procedure; request compatibility can be evaluated directly over its bounded witness context, without requiring memoization.

Lemma A.4 (Deletion complexity). *Under the stated search and dictionary models, the expected work of one local deletion is*

$$O(M \cdot efc \cdot (M + efc \cdot m) \log n) . \quad (1)$$

Proof. *Search and discovery.* IP-DiskANN's Algorithm 5 searches with the old vector and checks examined adjacency lists for incoming edges [1]. By the adopted model, the constant number of layer searches and incoming-edge scans costs $O(M \cdot efc \log n)$. At most $O(efc \log n)$ incoming sources are discovered. The outgoing pass issues $O(cM)$ proposals, but their source IDs all come from the same first k_r retained candidates, so it adds only $O(1)$ distinct source rows. Proposals are merged by source and layer, and each repair plan executes once. Thus the number r_a of reconstructed adjacency rows across both layers is $O(efc \log n)$. This bounds discovered repair rows, not the deleted point's full in-degree.

Work per repaired row. Each source receives $O(M)$ old targets, at most c incoming-side replacements, and $O(M)$ outgoing-side targets, giving an $O(M)$ target pool. Selecting replacements scores at most the first k_r retained candidates, rather than launching a fresh graph search for each source. Pool formation, duplicate checks, ranking, and RNG comparisons cost $O(M^2)$ per row. Here

pruning records direct witness IDs rather than propagating historical Markers; each pruned candidate is assigned to at most one owner. Coverage admission, old-Marker preservation, and direct-witness encoding therefore add $O(Mm)$. All base-layer repairs share the deletion's witness context, which contains at most efc candidates. For $O(M)$ owners, testing request compatibility and coverage against those witnesses costs $O(M \cdot efc \cdot m)$. The total per repaired row is consequently $O(M^2 + M \cdot efc \cdot m)$.

Context and nonincident Marker cleanup. Encoding the retained candidates and building their support lists costs $O(efc \cdot m)$. Cleanup applies to at most efc retained source rows. Each such source inspects $O(M)$ neighbors to select one dominator, then checks that owner's removable bits against at most efc witnesses. This costs $O((M + efc)m)$ per source and $O(efc(M + efc)m)$ in total.

Aggregation. Identifier lookup and work-list management use the stated expected constant-time dictionary model; target-pool edits have already been charged above. Publishing each row and its Markers costs $O(Mm)$, also covered by the per-row bound. There are no concurrent point workers or deferred inter-point repair searches for this single-point procedure. Writing T_a for the expected local work and substituting $r_a = O(efc \log n)$ gives

$$\begin{aligned} T_a &= O\Big(M \cdot efc \log n + efc(M + efc)m \\ &\qquad + (1 + r_a)(M^2 + M \cdot efc \cdot m)\Big) \\ &= O\Big(M \cdot efc \cdot (M + efc \cdot m) \log n\Big), \end{aligned}$$

which proves Equation (1). □

For fixed M, efc, and m, this is $O(\log n)$. As in the construction analysis, this local-algorithm bound uses the stated search and dictionary models.

Reconnection details. Each incoming- or outgoing-side proposal ranks the original first $k_r = 50$ retained candidate positions and admits at most $c = 3$ eligible endpoints. Invalid or retired endpoints, the deleted point, and the affected endpoint are excluded without widening that prefix. The deleted point's native placeholder still consumes its original rank.

RNG dominance and Marker cleanup. On a Marker-bearing layer, let $F(x) = \texttt{MEncode}(x.A, C)$ under the fixed Codebook C, and let *Cand* denote the retained layer-search candidates. For source u, neighbor v, and candidate w, define

$$D(u, v, w) \equiv dis(u, v) \le dis(u, w) \wedge dis(v, w) < dis(u, w).$$

The first comparison captures the distance ordering in RNG pruning; the second is its domination test. Nonincident Marker cleanup applies only to retained live candidate sources; examined-only rows may receive graph repair without this cleanup. For a deleted point a and each such source u, select the live neighbor v closest to u among those satisfying $D(u, v, a)$. If no such neighbor exists, no nonincident Marker is cleaned for that source. Equal-distance ties between valid cleanup owners follow adjacency-row order. The candidate bits for clearing are

$$e_{(u,v)}.\text{Marker} \wedge F(a) \wedge \neg F(v).$$

Clear a set bit b of this mask only when no live $w \in Cand \setminus \{a, u, v\}$ has both $F(w)[b] = 1$ and $D(u, v, w)$. Thus the target's own encoding and bits supported by retained live witnesses are protected. Reconstruction uses the same $D(u, v, w)$ when testing whether a selected owner already covers $F(w)$ and when choosing a compatible requesting owner for w. This is a current geometric compatibility test, not a recovery of exact historical pruning assignments.

Batched deletion and cross-point repair. The main paper's Marker-aware Deletion algorithm describes one logical point deletion. The batched implementation may execute several such point tasks in parallel. A repair can encounter another target that is being retired by a different task. It hands the source ID, layer, and optional Marker-recovery request to the active deletion task for that target. The recipient applies incoming repair deltas using its retained layer candidates. Closing a task is synchronized with outstanding producers so that reserved requests are not lost. Valid requests that arrive after the recipient has closed enter a batch-local deferred queue; completion epochs distinguish overlapping work from already-completed stale edges.

Deferred repairs may search again at width efc to construct a fresh bounded witness context and can generate further repair handoffs. Each producing worker services deferred work before returning, and the batch waits for its workers and their claimed repairs to finish. These repairs complete within the deletion call rather than waiting for a subsequent query. Liveness is rechecked when using witnesses and publishing edges. Callers must prevent external queries and mutations from overlapping this stop-the-world operation; parallelism is internal to the deletion batch. The single-point complexity bound above does not include this cross-point coordination or its additional repair searches.

A.2 Theoretical Analysis of Marker False Positives

Definition A.5 (Expected Case-(1) False Positive Rate). Consider a directed edge $e_{(u,v)}$ whose Marker is constructed by aggregating the attribute bitmaps of the dominating node v and the set of nodes dominated by v with respect to u, denoted by $D(e)$; hence $v \notin D(e)$.

Let R be a query predicate with selectivity sel, i.e., for a randomly drawn node p,

$$\Pr[p \models R] = \text{sel},$$

and assume that, conditional on the graph, predicate satisfaction events are independent and identically distributed across nodes.

A *Case-(1) false positive* occurs on edge e if the Marker is activated due to at least one dominated node satisfying the predicate, while the target node itself does not satisfy the predicate. Formally,

$$\text{FP}_{\text{case1}}(e) \iff \big(v.A \not\models R\big) \wedge \Big(\exists p \in D(e) : p \models R\Big).$$

This event requires a predicate-matching dominated node; it does not count aggregate matches assembled from different nodes across predicate components.

Theorem A.6 (Expected Case-(1) False Positive Rate). *Let $|D(e)|$ denote the dominate size of edge e, and let*

$$\mu = \mathbb{E}[\,|D(e)|\,]$$

be the average dominate size over all edges.

Under the assumptions of Definition A.5, the expected false positive rate induced by Case (1) is given by

$$\mathrm{FPR}_{\mathrm{case1}} = (1-\mathrm{sel}) \cdot \Big(1 - \mathbb{E}\big[(1-\mathrm{sel})^{|D(e)|}\big]\Big).$$

Moreover, the false positive rate admits the following upper bound that depends only on the average dominate size μ:

$$0 \;\le\; \mathrm{FPR}_{\mathrm{case1}} \;\le\; (1-\mathrm{sel}) \cdot \big(1 - (1-\mathrm{sel})^{\mu}\big).$$

In particular, under the common average-size approximation $|D(e)| \approx \mu$, we obtain

$$\mathrm{FPR}_{\mathrm{case1}} \;\approx\; (1-\mathrm{sel}) \cdot \big(1 - (1-\mathrm{sel})^{\mu}\big).$$

Proof. By definition, a Case-(1) false positive occurs when the Marker of an edge $e = (u \to v)$ is activated due to at least one dominated node satisfying the query predicate, while the target node v itself does not satisfy the predicate. Formally, the event is

$$\mathrm{FP}_{\mathrm{case1}}(e) \iff \big(v.A \not\models R\big) \;\wedge\; \Big(\exists p \in D(e) : p \models R\Big).$$

Conditioned on a fixed dominate size $|D(e)| = k$, independence implies

$$\Pr[v.A \not\models R] = 1 - \mathrm{sel},$$

and

$$\Pr\big(\exists p \in D(e) : p \models R\big) = 1 - \Pr\big(\forall p \in D(e) : p \not\models R\big) = 1 - (1-\mathrm{sel})^k.$$

Since the two events are independent, we obtain

$$\Pr(\mathrm{FP}_{\mathrm{case1}} \mid |D(e)| = k) = (1-\mathrm{sel}) \cdot \big(1 - (1-\mathrm{sel})^k\big).$$

Taking expectation over the random variable $|D(e)|$ yields

$$\Pr(\mathrm{FP}_{\mathrm{case1}}) = (1-\mathrm{sel}) \cdot \Big(1 - \mathbb{E}\big[(1-\mathrm{sel})^{|D(e)|}\big]\Big).$$

To derive a bound that depends only on the average dominate size $\mu = \mathbb{E}[|D(e)|]$, observe that the function $f(k) = (1-\mathrm{sel})^k$ is convex for $k \ge 0$. By Jensen's inequality,

$$\mathbb{E}\big[(1-\mathrm{sel})^{|D(e)|}\big] \;\ge\; (1-\mathrm{sel})^{\mathbb{E}[|D(e)|]} = (1-\mathrm{sel})^{\mu}.$$

Substituting this inequality gives

$$\mathrm{FPR}_{\mathrm{case1}} \le (1-\mathrm{sel}) \cdot \big(1 - (1-\mathrm{sel})^{\mu}\big).$$

Non-negativity of probabilities establishes the lower bound. □

Example. At sel = 1%, the upper bound in Theorem A.6 is approximately 1.97% for $\mu = 2$ and 2.94% for $\mu = 3$. At sel = 50%, the corresponding bounds are 37.5% and 43.75%. These are upper bounds for a varying dominated-set size; the exact rate equals them when every edge has the stated dominated-set size.

Next, we isolate Codebook discretization for conjunctive predicates on single-valued attributes. This model excludes dominated-node aggregation and set-valued categorical predicates. Let $a_j(v)$ denote the unique value of attribute $A^{(j)}$ in record v.

Let $R = \bigwedge_{j=1}^{m} R_j$ be a conjunctive predicate on m attributes, and let

$$\mathcal{R} \;=\; \{v \in \mathcal{D} \mid R(v) = \mathrm{true}\}$$

denote the set of records satisfying R.

Let $\widetilde{R}_j$ be the union of all Codebook regions containing a value that satisfies R_j. The Codebook-only approximation accepts the records

$$\widetilde{\mathcal{R}} = \{v \in \mathcal{D} \mid a_j(v) \in \widetilde{R}_j \text{ for all } j\}.$$

Thus $\mathcal{R} \subseteq \widetilde{\mathcal{R}}$. This is not the acceptance set of full, neighborhood-aggregated edge Markers.

Define the Case-(2) false positive rate as

$$\mathrm{FPR}_{\mathrm{case2}}(R) = \frac{\Pr[v \in \widetilde{\mathcal{R}} \setminus \mathcal{R}]}{\Pr[v \in \widetilde{\mathcal{R}}]}.$$

Theorem A.7 (Bounding Case-(2) False Positives). *Assume that:*

(i) *For each attribute $A^{(j)}$, the Codebook partitions its domain into s disjoint regions, each with probability mass at most $1/s$;*
(ii) *For each attribute $A^{(j)}$, the predicate R_j partially intersects at most b_j Codebook regions.*

If the joint predicate selectivity satisfies $\Pr[v \in \mathcal{R}] \ge \mathrm{sel} > 0$, then

$$\mathrm{FPR}_{\mathrm{case2}}(R) \;\le\; \frac{\sum_{j=1}^{m} \frac{b_j}{s}}{\mathrm{sel} + \sum_{j=1}^{m} \frac{b_j}{s}}.$$

Theorem A.8 (Codebook size for controlling Case-(2) false positives). *Under the assumptions of Theorem A.7,*

$$\mathrm{FPR}_{\mathrm{case2}}(R) \;\le\; \frac{\sum_{j=1}^{m} \frac{b_j}{s}}{\mathrm{sel} + \sum_{j=1}^{m} \frac{b_j}{s}}.$$

For a target $0 < \mathrm{FP} < 1$, to guarantee $\mathrm{FPR}_{\mathrm{case2}}(R) \le \mathrm{FP}$, it suffices that

$$s \;\ge\; \frac{(1-\mathrm{FP})}{\mathrm{FP} \cdot \mathrm{sel}} \cdot \sum_{j=1}^{m} b_j.$$

Proof. For a randomly drawn record v, let E_j be the event that $a_j(v) \in \widetilde{R}_j$ but $R_j(a_j(v)) = \mathrm{false}$. Such a value must lie in a Codebook region that intersects the satisfying set of R_j but is not fully contained in it. There are at most b_j such regions, each of probability mass at most $1/s$, so

$$\Pr[E_j] \le \frac{b_j}{s}.$$

Since

$$\widetilde{\mathcal{R}} \setminus \mathcal{R} \;\subseteq\; \bigcup_{j=1}^{m} E_j,$$

the union bound gives

$$\Pr[v \in \widetilde{\mathcal{R}} \setminus \mathcal{R}] \;\le\; \sum_{j=1}^{m} \Pr[E_j] \;\le\; \sum_{j=1}^{m} \frac{b_j}{s}.$$

This step does not require independence among attributes. Write $t = \Pr[v \in \mathcal{R}] \ge \mathrm{sel} > 0$ and $\delta = \Pr[v \in \widetilde{\mathcal{R}} \setminus \mathcal{R}]$. Since $\mathcal{R} \subseteq \widetilde{\mathcal{R}}$,

$$\Pr[v \in \widetilde{\mathcal{R}}] = t + \delta.$$

The ratio $\delta/(t+\delta)$ increases with δ and decreases with t. Using the preceding bounds gives

$$\mathrm{FPR}_{\mathrm{case2}}(R) = \frac{\delta}{t+\delta} \;\le\; \frac{\sum_{j=1}^{m} \frac{b_j}{s}}{\mathrm{sel} + \sum_{j=1}^{m} \frac{b_j}{s}}.$$

Requiring the right-hand side to be at most FP gives

$$(1-\mathrm{FP}) \sum_{j=1}^{m} \frac{b_j}{s} \;\le\; \mathrm{FP} \cdot \mathrm{sel},$$

which is equivalent to the stated sufficient condition on s. □

Example. Consider a conjunction $R = \bigwedge_{j=1}^{m} R_j$ on m single-valued attributes. If each R_j is a 1D range predicate under a contiguous Codebook partition, then $b_j \le 2$. Then $\sum_{j=1}^{m} b_j \le 2m$, and Theorem A.8 gives the sufficient condition

$$s \;\ge\; \frac{(1-\mathrm{FP})}{\mathrm{FP}\cdot\mathrm{sel}}\cdot 2m.$$

For instance, with $m = 4$, sel = 50%, and FP = 50%, it suffices to use $s \ge 16$ bits per attribute. If sel = 10% and FP = 50%, the bound becomes $s \ge 80$.

These theorems establish the computational and theoretical foundations of EMA. We analyze the index-time and space complexity of EMA, and provide a principled characterization of false positives arising from dominance-based aggregation (Case 1) and codebook granularity (Case 2). Together, these results clarify when marker-based pruning is effective and show how appropriate codebook sizing can bound granularity-induced false positives while preserving efficient graph traversal.

B DYNAMIC EVALUATION PROTOCOL

The main paper evaluates insertion, synchronous deletion, and complete-record replacement on SIFT10M. Insertion grows a 5M-record checkpoint to 10M in five 1M-record stages. Deletion begins from that exact insertion-final checkpoint and follows a fixed random permutation (seed 12345), removing five 1M-record batches. Replacement starts from the unchanged initial 5M checkpoint; each stage synchronously deletes 1M original records and inserts complete replacements at fresh labels, without slot reuse. Both vectors and attributes change, and the live set remains at 5M records. Graph and Marker repair finish before queries; there is no query-triggered patch or final rebuild.

Fixed configuration. Attribute Codebooks and the representative level plan are prepared offline from the full 10M source and held fixed across stages and traces. All traces reuse the first 1,000 query vectors and fixed predicates $((a_{\mathrm{num}} \in [\ell_i, h_i]) \wedge (9 \in A_{\mathrm{cat}})) \vee (12 \in A_{\mathrm{cat}})$, where a_{num} and A_{cat} are a record's numerical value and categorical label set. Numerical bounds are inclusive and are not retuned after mutation. Mean selectivity is 9.79% on the full corpus and may vary with the live set. Construction and deletion search use $M = 40$ and $efc = 300$, with 128 Marker bits per attribute and $ef_{\mathrm{top}} = 1$. Marker checking and routing are enabled, with routing minimum degree 8 and no tail backfill. Construction, maintenance, and exact-GT computation use 32 threads on one NUMA node; queries use one physical core and $k = 10$.

Query timing. We reload each saved stage checkpoint and compute exact filtered ground truth over its live set. The native implementation, graph contents, predicates, and routing parameters are unchanged during query-only evaluation. We calibrate ef by recall alone and freeze the adjacent recall bracket before timing. After warmup, current and initial graphs are measured in paired rounds on the same physical core. At each endpoint, we take median QPS over the first seven CPU-clean complete rounds, then linearly interpolate QPS against recall at 95%. All 18 stage checkpoints support a measured 95% recall bracket; no minimum-ef substitute or extrapolated value is used.

Table 1: Per-dataset breakdown of the main paper's Figure 5. Each row equally weights six selectivities; ratios are dimensionless multiples.

Workload	Dataset	Min–max	Mean	Wins/total
L+R (high)	RedCaps	2.32–3.71	2.72	6/6
L+R (high)	SIFT	3.13–9.09	5.14	6/6
L+R (high)	YouTube	6.05–8.69	7.14	6/6
L+R (high)	Wiki	2.84–13.01	4.88	6/6
L+R (low)	RedCaps	3.28–5.86	3.88	6/6
L+R (low)	SIFT	9.09–15.41	11.39	6/6
L+R (low)	YouTube	1.20–6.08	4.07	6/6
L+R (low)	Wiki	1.41–1.98	1.68	6/6
Composed	RedCaps	2.43–9.53	4.74	6/6
Composed	SIFT	1.70–28.18	12.25	6/6
Composed	YouTube	1.24–6.73	4.88	6/6
Composed	Wiki	3.91–6.99	5.47	6/6

Mutation timing. Mutation-call wall times are retained from the original execution producing these checkpoints; mutations are not repeated or their times rescaled for the 95% recall evaluation. Each time covers one batch of 1M logical records, not cumulative work. Timed calls include synchronous graph/Marker maintenance and binding or source-row preparation. Deletion also includes batch-end adjacency cleanup and entry-point bookkeeping, beyond the local procedure analyzed in the main paper. Initial preparation and construction, exact GT, recall calibration, query timing, and checkpoint I/O are excluded. For replacement, deletion and insertion times are summed for one 1M-replacement batch, not divided by 2M operations.

C SPEEDUP ACCOUNTING AND REPRODUCIBILITY

The main paper's headline covers its Figure 5 only: three panels, four datasets, and six selectivities per dataset, totaling 72 configurations. For each configuration $c = (\text{panel, dataset, selectivity})$, we compute

$$r_c = \frac{\mathrm{QPS}_{\mathrm{EMA},c}}{\max_{b\in B_c} \mathrm{QPS}_{b,c}}, \qquad \bar{r} = \frac{1}{|C|}\sum_{c\in C} r_c,$$

where B_c contains only positive, finite, displayed baseline points reported at the same 95% recall@10 target. The candidate methods are NaviX, ACORN, VBase, Milvus, iRangeGraph, and Filtered DiskANN, when actually displayed in that configuration. Milvus is included consistently as an internally multi-threaded system baseline; these are not uniformly single-thread algorithmic comparisons. Missing or lower-recall points are not interpolated, substituted, or assigned zero QPS. Tied fastest baselines are all recorded. Every paired configuration has equal weight, and a win means $r_c > 1$ before rounding. Ranges are extrema of pointwise ratios, not extrema of per-dataset means. The headline is 1.20×–28.18×, arithmetic mean 5.69×, with 72/72 wins.

The two label+range panels both display 10% selectivity. We retain these as panel-specific configurations rather than deduplicating or averaging their QPS. Their source records differ: EMA is 1,577 versus 1,577.67 QPS on RedCaps and 507.95 versus 814 QPS on Wiki. The query-cohort/run identity of these endpoints still requires confirmation; the 72 configurations are not 72 independent repetitions. The RedCaps ACORN value 51.15 QPS at this boundary is tagged 95% in the high-selectivity source but 90% in the low-selectivity source. It is conservatively excluded from both panels' eligible baselines pending recall-log confirmation; this does not change the selected fastest baseline.

The static arrays record recall targets, not the achieved recall of every QPS measurement. The audit therefore checks consistency with reported targets; independent verification of target attainment requires the original recall logs. Recall@100 curves compare EMA at approximately 99% recall with other methods at 95% and are excluded from speedup aggregation. The five negatively correlated configurations have no qualifying 95%-target baseline and remain unpaired, not losses or infinite speedups. Cumulative-ablation curves use different recall grids; no interpolated speedup summary is inferred from them.

For range-only queries, EMA exceeds NaviX at their common plotted 95%-target points, but does not exceed iRangeGraph at any of their common plotted 95%-target configurations. At 2% selectivity, iRangeGraph has no plotted result, and EMA exceeds the other qualifying baselines. The YouTube-RGB difference at this selectivity is 130.96 versus 130.95 QPS, which does not establish a meaningful performance advantage.

All reported static ratios and parameter-comparison ratios are generated by running `Rscript static_summary.R` from `paper/CH5/figures`. The script loads the same six inline datasets as the main paper's plotting generators; it does not use the unavailable alternative CSV referenced by `static_ema_data.R`. The following files are under `paper/CH5/figures/data`:

- `ema_static_plot_points.csv` records every finite plotted measurement and its eligibility/exclusion reason.
- `ema_static_pointwise_speedups.csv` records each selected baseline and unrounded ratio.
- `ema_static_speedups.csv` records all scope/dataset aggregates.
- `range_range_plot_points.csv` archives the supplementary range+range inputs.
- `parameter_sensitivity.csv`
  Measurements for both parameter studies.

The latter two inputs were transcribed from the original plotting scripts whose PDFs match this report's assets. The script also generates the shared LaTeX numbers and table rows and recomputes replacement retention from `ema_dynamic_v11.csv`. Source-record conflicts and their disposition are listed in `ema_speedup_audit_issues.csv`. Ratios and percentages are rounded to two decimals only for display (three for a positive ratio below 0.01); aggregates and win counts use unrounded values. No statistical significance is inferred from a win count.

Algorithm 1: `Full Marker-augmented Pruning`

Input: Node u; distance-sorted $Cand$ with $|Cand| \geq M$; layer budget M; coverage-check bypass g; Codebook C; coverage threshold M_{div}

Output: Neighbor list $Nbrs$; Markers on edges incident to u

$Nbrs \leftarrow \emptyset$;
$CT \leftarrow \mathbf{0} \in \mathbb{N}^{m \times s}$;
foreach $v \in Cand$ **do**
  $dom \leftarrow$ false;
  foreach $w \in Nbrs$ **do**
    if $dis(w, v) < dis(u, v)$ **then**
      if $e_{(u,v)}$ *is an old edge* **then**
        $e_{(u,w)}$.Marker $|=$ $e_{(u,v)}$.Marker;
      else
        $e_{(u,w)}$.Marker $|=$ `MEncode`$(v.A, C)$;
      $dom \leftarrow$ true;
      break;
  if dom = false **then**
    if $|Nbrs| = M$ **then break**;
    if $|Nbrs| < g$ ***or*** `LowCoverage`$(u, v, CT, C, M_{\text{div}})$ **then**
      $Nbrs \leftarrow Nbrs \cup \{v\}$;
      $\mathbf{z} \leftarrow$ `MEncode`$(v.A, C)$;
      $e_{(u,v)}$.Marker $|=$ $\mathbf{z}$;
      increment $CT_{j,C_j(x)}$ for each $x \in v.A^{(j)}$, all j;

D COMPLETE CONSTRUCTION POLICY

The main paper's pruning pseudocode presents the core Marker-propagation procedure. The experimental construction policy additionally uses the diversity-aware admission rule in Algorithm 1; component controls selectively disable this rule. This section records the complete policy underlying the reported measurements.

Source-relative coverage admission. For a nondominated candidate, `LowCoverage` tests counts of attribute-value occurrences among retained targets. For a numerical attribute, it sums CT over the inclusive bucket interval between u and v. For a categorical attribute, it checks bucket counts for labels in $u.A^{(j)} \cap v.A^{(j)}$. The test succeeds if any such interval or shared-label bucket has count below M_{div}. We bypass this coverage test while $|Nbrs| < g$, where $g = \max(1, \lfloor M_{\text{HNSW}}/3 \rfloor)$ depends on the configured HNSW degree, not the layer capacity M. With $M_{\text{HNSW}} = 40$, $g = 13$ and the bottom-layer capacity is $M = 80$. Unless stated otherwise, $M_{\text{div}} = 16$.

Each admitted target contributes its own encoding to the edge Marker and increments CT once per attribute value; labels mapped to the same bucket contribute separately. At capacity, dominated candidates may still contribute Markers until the next nondominated candidate terminates pruning. If $|Cand| < M$, all candidates are retained, with target-own initialization for new edge Markers. Deletion row reconstruction uses the same layer-configured admission rule while recording direct witnesses instead of propagating historical Markers.

E COMPONENT ABLATION ACROSS SELECTIVITIES

Configuration and controls. The expanded study uses YouTube-RGB's 1M unnormalized 1024-dimensional vectors, 1,000 queries, and mixed numerical-range and categorical-label predicates. Nominal selectivities are 1%, 2%, 3%, 5%, 7%, 10%, 20%, 40%, 60%, 80%, and 100%. Mean realized selectivity ranges from 0.995% to 100%; the nominal 5% and 80% cells realize approximately 5.31% and 81.04%, respectively. The configuration is $M_{\text{HNSW}} = 40$, $efc = 300$, 128 Marker bits per attribute, $d_{\min} = 16$, and upper-layer search width 1. Construction uses 32 threads and queries use one thread; the native seed is 100. Inner product is evaluated without vector normalization.

`EMA` enables diversity-aware pruning, Marker-guided expansion, and bounded edge recovery. `Diversity only` retains diversity-aware pruning but disables Marker checks and recovery.

`No Recovery` and `No Diversity` disable the named component relative to `EMA`. The `Post-filtering` control shares the implementation and edge-Marker storage layout. Coverage admission, Marker checks, and recovery are disabled. This control is not an independently built upstream HNSW binary. One graph is built for each admission policy with the same inputs, Codebook, insertion IDs, level plan, and seed. Parallel construction interleavings are not deterministic, so comparisons across those two graphs are not repeated-build estimates. `Diversity only`, `No Recovery`, and `EMA` instead share the identical saved diversity-aware graph. The main paper combines the component study's `Post-filtering` curve with the fixed-graph run's strict in-filtering, `No Recovery`, and `EMA` curves in one throughput panel. Its work panel uses `Diversity only` as same-graph post-filtering: Marker checks and recovery are disabled on `EMA`'s saved graph. Figure 1 retains all five component variants.

Fixed-graph traversal comparison. A separate run freshly pairs strict in-filtering, `No Recovery`, and `EMA` on this shared graph. All policies use the same unfiltered upper-layer navigation and initial seed. For newly reached bottom-layer neighbors, strict in-filtering requires an exact predicate match before distance evaluation and queue insertion. `No Recovery` checks the edge Marker instead and disables recovery; `EMA` additionally enables recovery. This comparison isolates query policies from graph construction, but does not separately isolate transferred evidence from target-only Marker encoding.

Recall and timing accounting. The target is 95% recall@10. Search width is swept and the first sampled target crossing is refined within its bracket, up to $ef = 4096$. QPS at the exact target is interpolated only between timed measurements bracketing that target. Each QPS point is the median of seven accepted paired timing rounds, and each reported speedup is the median of the corresponding seven per-round ratios. This interpolation policy is specific to the expanded ablation and does not change the main Figure 5 accounting. The two studies are timed independently; their QPS values are not combined into cross-run ratios. Query batches that return fewer than ten results are marked incomplete: no batch recall, zero QPS, or interpolated target value is assigned. At 1%–10%, strict in-filtering and `No Recovery` fail to complete a batch across the tested search widths. At 20%, `No Recovery` completes and reaches the target, while strict in-filtering still encounters incomplete batches. Failure witnesses establish that a query can terminate with insufficient results; they do not estimate a workload-wide failure rate or a recall ceiling.

Observed component roles. At 1%–10%, `EMA` achieves 1.37×–1.55× the `Post-filtering` control's throughput. Diversity-aware pruning is not uniformly beneficial, and removing it from `EMA` can improve throughput. Without recovery, Marker-guided expansion achieves 2.34×, 1.60×, and 1.16× the QPS of strict in-filtering at 40%, 60%, and 80%, respectively. These gains establish a role for Marker-guided expansion independent of recovery. Recovery enables complete target-recall queries under more selective predicates but need not improve throughput when Marker-only expansion already suffices. At 100%, filtering rejects no directions and Marker checking has no filtering benefit.

Diagnostic work. Counters are collected in untimed replays at each policy's first sampled 95%-recall crossing, not at the interpolated point. For `EMA` versus `Diversity only` at 1%, the respective search widths are 32 and 17, with observed recall 0.9500 and 0.9509. Distance evaluations decrease from 29,610.589 to 16,087.146 per query (45.67%), and exact predicate checks decrease from 3,945.847 to 729.552 (81.51%). Expanded bottom-layer nodes increase from 1,706.437 to 2,064.040 (20.96%). Thus increased node expansion can coexist with lower total distance and predicate work. The replays match timed query outputs but do not attribute wall time to a particular hardware bottleneck.

Reproducibility. The CSV snapshots preserve summaries, paired contrasts, and timed recall–QPS curves. They are stored under `paper/CH5/figures/data`:

- `ablation_components_*`: component-study CSVs.
- `ablation_fixed_graph_*`: fixed-graph CSVs.
- `ablation_provenance.json`: source runs, hashes, configuration, and graph-control semantics.

The generator `ablation_plot.R` writes the figure.

Numerical macros are exported to `ema_ablation_summary.tex`.

Original cumulative ablation. Figure 2 retains the earlier RedCaps 10% and YouTube-RGB 1% recall–QPS curves. `No Recovery` trails `Diversity only` over their overlapping RedCaps recall range, while adding recovery improves throughput there. The expanded study varies selectivity within YouTube-RGB; it does not establish the cause of the RedCaps result. These original curves use different recall grids and are not included in the expanded paired speedup calculations.

F LOW SELECTIVITY PERFORMANCE UNDER BIASED SELECTIVITY DISTRIBUTIONS.

In practice, multi-predicate queries do not always exhibit evenly distributed selectivities across predicates. We therefore evaluate performance under biased selectivity distributions for range+range queries. Figure 3 reports the results at 95% recall, where the selectivity of the first predicate varies from 55% to 80%, while the selectivity of the second predicate ranges from 1.82% to 12.5%.

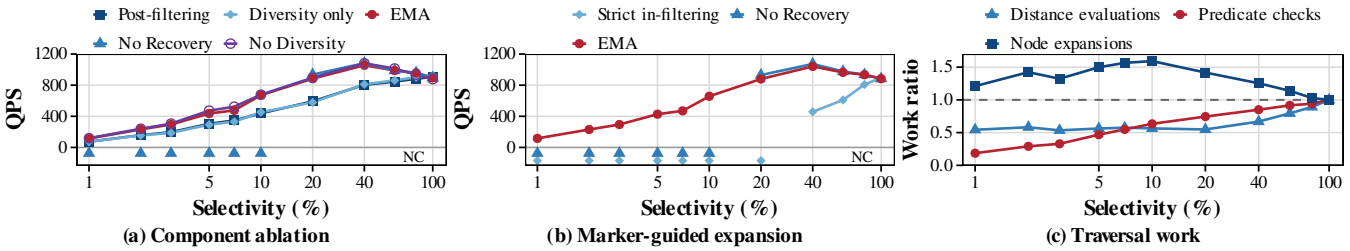


Figure 1: Expanded YouTube-RGB component study at nominal selectivities. QPS is evaluated at 95% recall@10; NC marks incomplete query batches. Work ratios compare diagnostic replays of `EMA` and `Diversity only` on their shared graph.

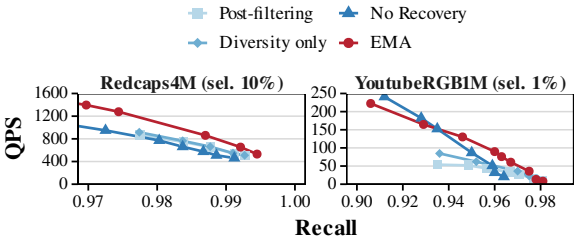


Figure 2: Original cumulative ablation on RedCaps (10%) and YouTube-RGB (1%).

Across the 24 plotted range+range configurations, EMA/best-baseline QPS ratios are 0.41×–8.78× (arithmetic mean 2.44×; 22/24 wins), using the same baseline-selection and equal-weight rules as above. These results describe this biased workload; they do not isolate the effect of selectivity allocation from other workload differences.

We additionally include `iRangeGraph` in this evaluation as a strong baseline for multi-predicate range queries. `iRangeGraph` benefits from a high-quality sub-index on the first attribute; however, when the first predicate becomes relatively loose, its search behavior increasingly resembles post-filtering, which limits its effectiveness compared to `EMA`. `iRangeGraph` failed to construct its index within the 20-hour time limit on Wiki and is not included there.

On YouTube-RGB, `Milvus` exceeds EMA at 1% and 2% selectivity, but not at the other four plotted selectivities. The small dataset allows Milvus to leverage parallel linear scans under highly restrictive predicates. This advantage reflects `Milvus`'s internal multi-threaded execution.

G PARAMETER SENSITIVITY ANALYSIS

G.1 Effect of Graph Degree M

We evaluate the impact of the maximum graph degree $M \in \{16, 32, 40, 64\}$. All other parameters are fixed (`ft_bits` = 128, `ft_routing_min_deg` = 8, $ef_c = 300$), using single-threaded execution over 1,000 queries. Table 2 reports QPS at 95% recall, where bold entries indicate the best M for each selectivity.

Overall, the optimal choice of M depends on selectivity. At low selectivity (1%–3%), smaller M performs best, as it reduces neighbor comparison cost and enables faster transition to edge recovery. Increasing M brings no benefit in this regime due to limited candidate expansion. At moderate selectivity (10%–40%), $M = 32$ achieves the highest QPS, balancing search quality and traversal overhead. At high selectivity (80%–100%), $M = 40$ becomes optimal, as slightly larger neighborhoods improve connectivity without incurring excessive scanning.

Notably, overly large M (e.g., 64) consistently underperforms due to increased traversal and unnecessary neighbor evaluation. These results suggest that moderate values of M provide the best trade-off between exploration cost and graph connectivity across selectivity levels.

Table 2: HashANN QPS at recall=0.95 on Redcaps_4M across selectivity and graph degree M.

M	1%	3%	10%	40%	80%	100%
16	**375**	**635**	931	2200	2958	3765
32	308	591	**1112**	**2606**	3097	4904
40	330	576	1073	2575	**3436**	**5438**
64	286	533	1056	2364	3399	4870
best M	16	16	32	32	40	40

G.2 Effect of FT Width (`ft_bits`)

We evaluate the impact of `ft_bits` on performance. Figure 4 reports QPS at 95% recall across selectivity.

`ft_bits`=128 achieves the highest QPS at all six plotted selectivities. Using 32 bits as the fixed reference at each selectivity, its QPS ratios are 1.08×–1.17× (arithmetic mean 1.13×; 6/6 wins), corresponding to 8.11%–17.05% higher QPS. Increasing from 32 to 128 bits is consistent with reducing false-positive routing, whereas 256 bits does not improve the plotted results and increases the summary footprint; the experiment does not isolate cache effects. The 64-bit variant remains below 128 bits at all six points. These results concern this parameter experiment, not the main paper's separate 256-bit static default.

G.3 Effect of Edge Recovery Threshold (`min_deg`)

We evaluate the effect of the minimum degree in edge recovery (`min_deg`). Figure 5 reports QPS at 95% recall under varying selectivity.

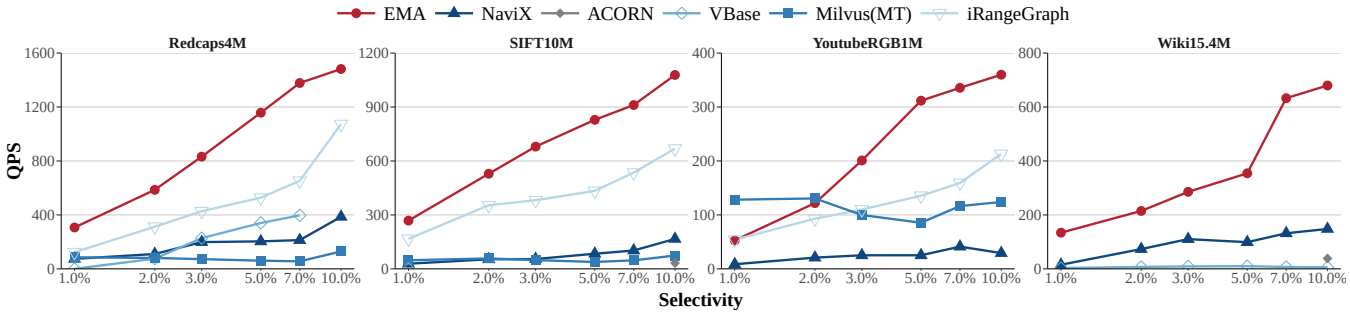


Figure 3: QPS for range+range biased multi-predicate query at 1%-10% selectivity with 95% recall@10.

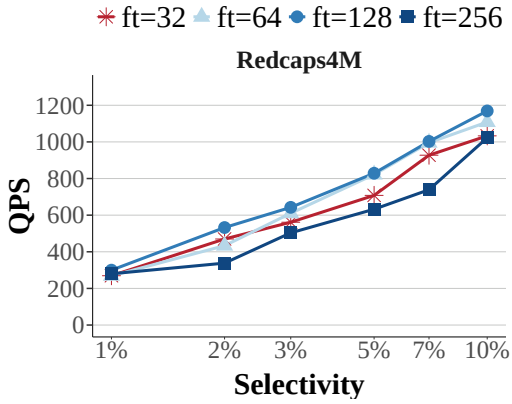


Figure 4: Impact of `ft_bits` on QPS at 95% recall across different selectivity levels.

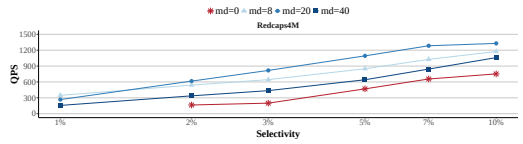


Figure 5: Impact of `min_deg` on QPS at 95% recall across different min_deg levels.

Among the tested thresholds $\{0, 8, 20, 40\}$, 8 is best at 1% selectivity and 20 is best at the other five selectivities; no threshold between 15 and 19 was tested. At each selectivity, we use the fastest tested threshold reaching the 95% target as the reference. Relative to that reference, threshold 40 loses 20.22%–54.96% QPS, where loss is $100(1 - \mathrm{QPS}_{40}/\mathrm{QPS}_{\mathrm{best}})$. The reciprocal throughput ratios, $\mathrm{QPS}_{\mathrm{best}}/\mathrm{QPS}_{40}$, are 1.25×–2.22× (arithmetic mean 1.73×; threshold 40 is slower in 6/6 configurations). This is a per-selectivity best-tested reference, not one globally optimal threshold. The results support bounded recovery but do not establish behavior for thresholds above 40, neighborhood saturation, or equivalence to post-filtering. Threshold 0 at 1% selectivity fails to reach the target and is excluded rather than assigned a throughput.

Table 3: FPR on Redcaps1M.

Selectivity	FPR	
	Range+Label	Range
1%	3.84%	2.99%
5%	4.80%	3.67%
10%	5.41%	4.27%
20%	6.20%	5.18%
60%	7.97%	7.71%

H OTHER STUDIES

H.1 False positive rate of the Marker.

The effectiveness of EMA depends on the accuracy of the Marker, as a lower false positive rate (FPR) reduces unnecessary candidate exploration during joint filtering. Table 3 reports the FPR on RedCaps1M across selectivity levels ranging from 1% to 60%.

The reported false positive rates are measured with edge recovery disabled and a fixed search parameter (efs), so as to isolate the accuracy of the Marker itself. Across this broad range of selectivities, the FPR consistently remains below 10% for both range+label and range-only queries. As selectivity increases, the FPR grows moderately, which is expected as filtering becomes less restrictive.

Overall, these results indicate that the Marker provides effective pre-checking with a low and stable false positive rate, supporting efficient query processing across different selectivity regimes.

H.2 Performance at Recall@100.

Figure 6 reports the QPS under the R@100 setting. While EMA exhibits slightly lower throughput in some cases, it is important to note that EMA consistently achieves a higher recall level (approximately 99%) compared to the 95% recall target used by other methods.

This difference arises from the more conservative convergence criteria adopted by EMA, which prioritize retrieval accuracy and result stability. As a result, these curves illustrate different recall operating points and are excluded from equal-recall speedup statistics.

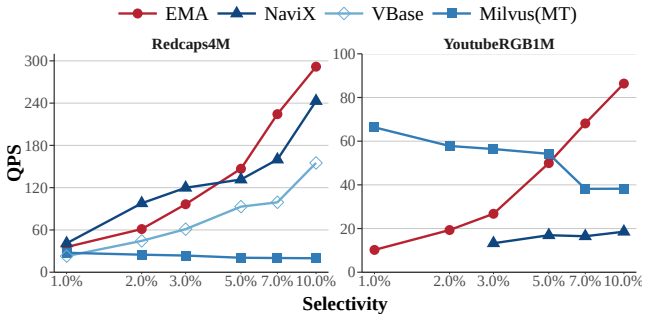


Figure 6: Label-range recall@100 results: EMA is reported at approximately 99% recall, versus 95% for the other methods. These curves are not equal-recall speedup comparisons.

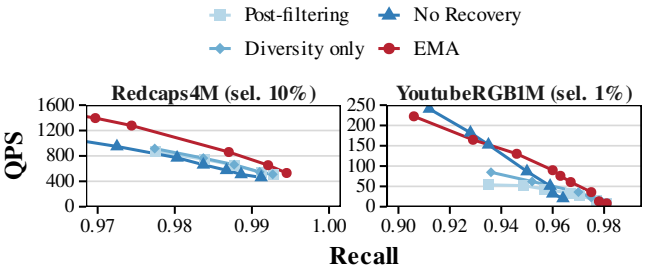


Figure 7: Ablation study on Redcaps and Youtube-RGB.

H.3 Ablation study.

Figure 7 shows cumulative additions of diversity-aware pruning, Marker-guided expansion, and bounded edge recovery. Queries combine range and label constraints at 1% selectivity on YouTube-RGB and 10% on RedCaps.

Diversity-aware pruning improves YouTube-RGB more than RedCaps. Adding Marker-guided expansion improves moderate-recall throughput on YouTube-RGB but can lose throughput at high recall; on RedCaps, `No Recovery` trails `Diversity only` over their overlapping recall range. Enabling recovery improves RedCaps throughout the plotted range and improves higher-recall YouTube-RGB performance relative to `No Recovery`, while adding overhead at lower recall. These results support complementary guidance and recovery, not a uniform gain from every component.

REFERENCES

[1] Haike Xu, Magdalen Dobson Manohar, Philip A. Bernstein, Badrish Chandramouli, Richard Wen, and Harsha Vardhan Simhadri. 2025. In-Place Updates of a Graph Index for Streaming Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2502.13826* (2025). https://arxiv.org/abs/2502.13826